

DAGFormer: Learned Computation Graphs for Transformers

Progress Report — March 2026

Abstract

This report summarizes our experiments on learning input-dependent computation graphs (DAGs) for transformer language models. We cover two lines of work: **midtraining** on a pretrained OLMo-1B (Section 2), and **pretraining from scratch** with a 300M model (Section 3). Midtraining failed because $\mathbf{A} = \mathbf{1}$ (dense connectivity) is a flat optimum for pretrained weights. Pretraining from scratch shows that self-embed predictors *can* learn meaningful sparse topologies (mean $\mathbf{A} \approx 0.42$), but a $3.5\times$ throughput overhead makes direct comparison unfair. We present five new strategies currently under evaluation and propose additional recipes for discussion.

1 Introduction

Standard transformers use a fixed sequential computation graph: layer $0 \rightarrow 1 \rightarrow \dots \rightarrow L$. Every input sees the same wiring regardless of content. **DAGFormer** replaces this with a learned, input-dependent directed acyclic graph (DAG) that controls which attention heads feed into which other heads.

The core modification: each attention head j in layer l receives a *gated combination* of prior heads' outputs:

$$\text{input}_j = \text{embedding} + \sum_{l' < l} \text{mlp}_{l'} + \sum_{i: \text{layer}(i) < l} \mathbf{A}[i, j] \cdot \text{head_output}[i]$$

where $\mathbf{A} \in [0, 1]^{N \times N}$ ($N = \text{layers} \times \text{heads}$) is block-upper-triangular. When $\mathbf{A}[i, j] = 1$ for all valid pairs, this reduces to the standard residual stream.

Oracle motivation. An oracle search (500 gradient steps per 1024-token window, directly optimizing $\mathbf{A}$) reduced NLL from 2.58 to 0.12 on OLMo-1B (median across 50 windows, 100% improved). This demonstrates enormous headroom but is too slow for practical use. DAGFormer aims to *amortize* this search into a learned predictor.

Architecture. The predictor maps input context to $\mathbf{A}$ via: context $\rightarrow$ encoder $\rightarrow$ MLP $\rightarrow$ low-rank $Z = UV^\top + b \rightarrow$ mask $\rightarrow$ Gumbel-Sigmoid $\rightarrow$ cascading gate $\rightarrow \mathbf{A}$. Training is end-to-end: NLL gradients flow through the modified OLMo forward back to the predictor. See [structure_predictor.html](#) for the full interactive architecture diagram.

2 Midtraining on Pretrained OLMo-1B

Our first approach: keep the pretrained OLMo-1B and learn topology on top of it. We explored **Phase 1** (frozen OLMo, only predictor trains) and **Phase 2** (unfrozen OLMo, joint training).

Both failed.

2.1 Phase 1: Frozen OLMo — $\mathbf{A} = 1$ Is a Flat Optimum

With OLMo frozen, any deviation from $\mathbf{A} = 1$ hurts. We tested multiple `init_logit` values controlling how far from dense the predictor starts:

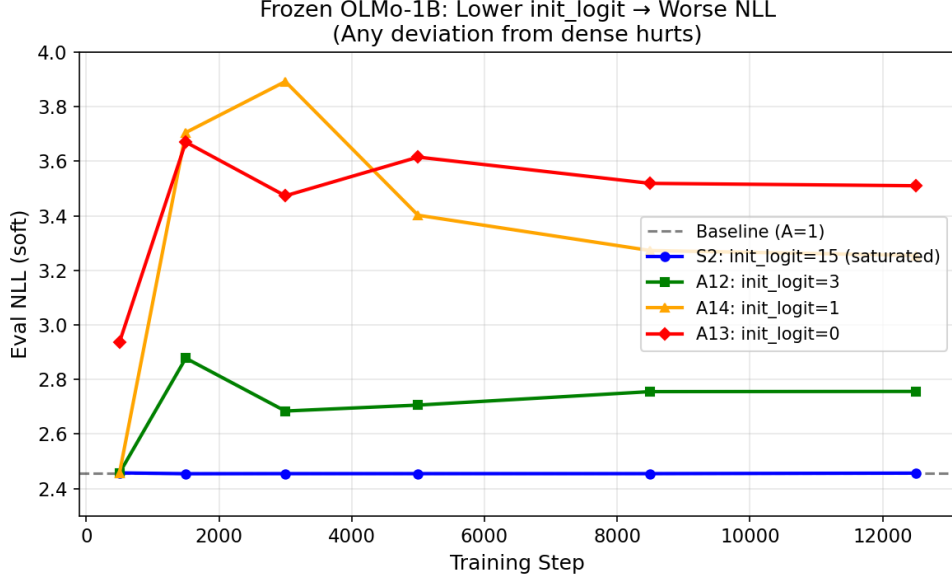


Figure 1: Frozen OLMo-1B: lower `init_logit` → further from dense → worse NLL. S2 (`init_logit=15`) stays at baseline but has saturated sigmoid (no gradient). A12/A13/A14 have gradient flow but converge to fixed topologies *worse* than dense.

To understand *why* $\mathbf{A} = 1$ is optimal, we computed $\partial\mathcal{L}/\partial\mathbf{A}$ at $\mathbf{A} = 1$ across 50 eval windows:

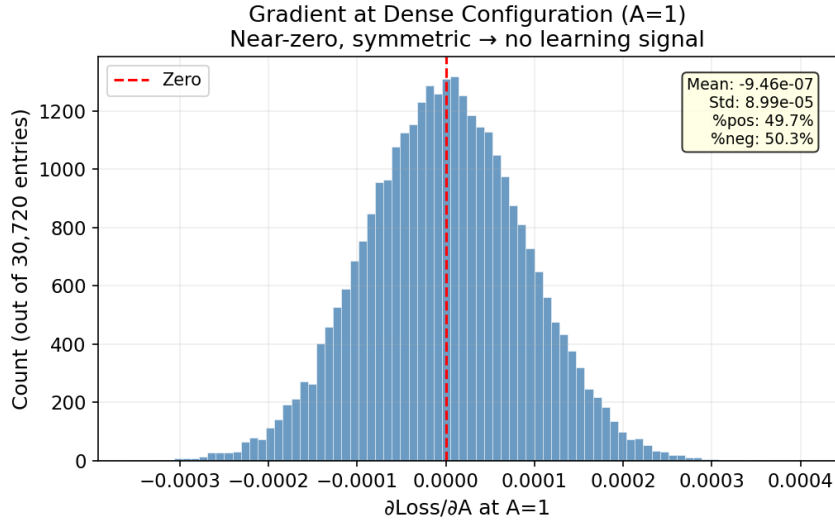


Figure 2: Gradient at $\mathbf{A} = 1$: symmetric around zero (49.7% positive, 50.3% negative, mean $\approx 10^{-6}$, std $\approx 9 \times 10^{-5}$). No consistent direction exists. Pretrained weights make $\mathbf{A} = 1$ a flat plateau.

2.2 Phase 2: Unfrozen OLMo — The LR Dilemma

Unfreezing OLMo allows the model to adapt to non-dense topologies. We ran 10 experiments varying OLMo LR, predictor LR, temperature schedule, initialization, and sparsity pressure.

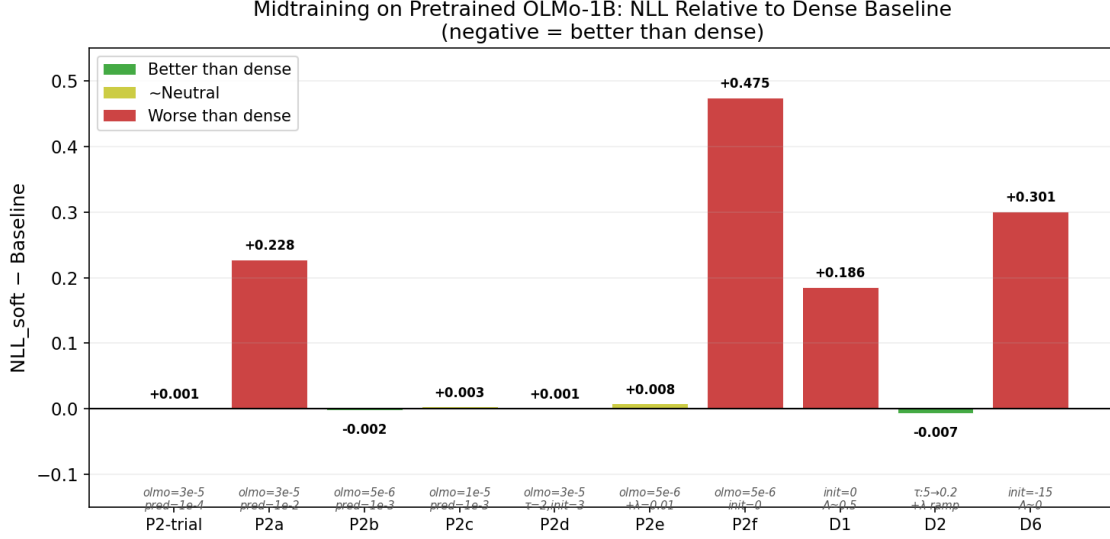


Figure 3: NLL relative to dense baseline for all 10 midtraining experiments. Only P2b (-0.002) and D2 (-0.007) show improvement; both remain near-dense (mean $\mathbf{A} > 0.97$). Most experiments are substantially worse.

The core dilemma:

- **High OLMo LR:** OLMo quickly adapts to dense via CPT. The predictor has no incentive to deviate from $\mathbf{A} \approx 1$.
- **Low OLMo LR:** OLMo cannot adapt to non-dense topologies. Any deviation from $\mathbf{A} = 1$ destroys information.
- **High predictor LR:** Topology collapses to random (P2a: mean $\mathbf{A} = 0.5$, NLL $+0.23$).
- **Starting sparse (D1, D6):** OLMo is damaged by missing connections it relies on.

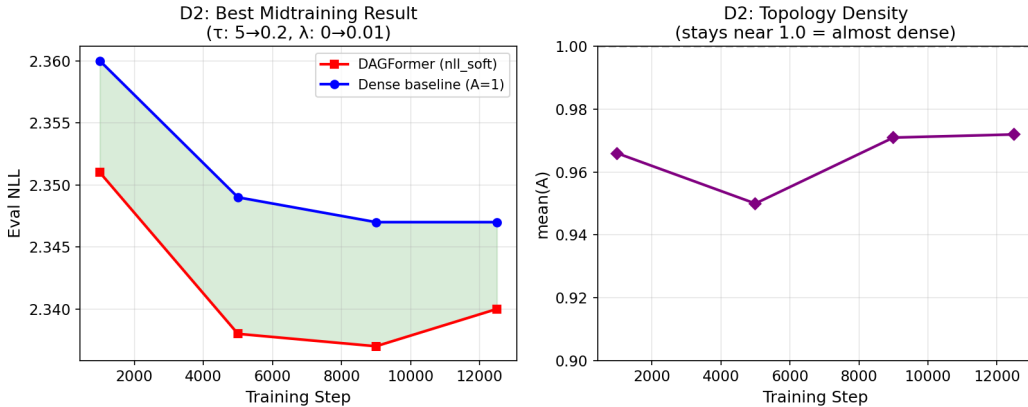


Figure 4: D2 ($\tau: 5 \rightarrow 0.2, \lambda: 0 \rightarrow 0.01$) achieved -0.007 NLL vs baseline, but topology stayed near-dense (mean $\mathbf{A} \approx 0.97$) and `jaccard_var`=0 throughout (no context-dependent routing). The improvement is statistically significant ($p < 10^{-6}$, $n = 500$ paired t -test) but tiny and context-independent.

2.3 Summary: All Midtraining Experiments

Exp	OLMo LR	Pred LR	Strategy	NLL-Base	mean A	Jaccard
P2-trial	3e-5	1e-4	Default	+0.001	0.98	0
P2a	3e-5	1e-2	High pred LR	+0.228	0.50	0
P2b	5e-6	1e-3	Low OLMo LR	− 0.002	0.99	0
P2c	1e-5	1e-3	Med OLMo LR	+0.003	0.99	0
P2d	3e-5	1e-3	Low τ +init	+0.001	0.99	0
P2e	5e-6	1e-3	+Sparsity	+0.008	0.97	0
P2f	5e-6	1e-3	init_logit=0	+0.475	0.56	0
D1	5e-6	1e-3	A starts ≈ 0.5	+0.186	0.71	0
D2	5e-6	1e-3	τ anneal+ λ	− 0.007	0.97	0
D6	5e-6	1e-3	A starts ≈ 0	+0.301	0.56	0

Table 1: All midtraining experiments on OLMo-1B. **Jaccard**=0 everywhere: the predictor produces identical topology for all inputs (no context-dependence).

2.4 Why Midtraining Failed

Pretrained weights are optimized for dense connectivity. The loss landscape at $\mathbf{A} = \mathbf{1}$ is a **flat plateau** with near-zero gradient ($\sim 4 \times 10^{-5}$), making gradient-based topology learning impossible. Joint training creates a chicken-and-egg problem: OLMo adapts to dense faster than the predictor can propose useful alternatives.

Conclusion: We need to train from scratch so that weights and topology co-evolve from random initialization.

3 Pretraining from Scratch — 300M OLMo-2

We switched to pretraining a 300M OLMo-2 (12 layers $\times$ 16 heads, $\mathbf{A} \in \mathbb{R}^{192 \times 192}$) from random init. Dense baseline: 10K steps on Dolma v1.7, 5.24B tokens, **eval NLL = 3.609**, throughput 81K tok/s ($4 \times A40$).

3.1 DAGFormer Variants (Batch 1: From Scratch)

Three predictor types, all from random init with the same 5.24B token budget:

Variant	Predictor	tok/s	Steps	NLL (soft)	Own baseline	mean A
Baseline	None (dense)	81K	10000	3.609	—	1.0
Static	Learnable $UV^\top$	23K	~ 4690	~ 5.89	~ 5.98	0.98
Self-embed	Mean-pool embed	23K	~ 4670	~ 5.75	~ 5.96	0.42
Qwen	Frozen Qwen-0.6B	14K	~ 2920	~ 6.15	~ 6.28	0.41

Table 2: From-scratch pretraining results. All DAGFormer variants see $\sim 3.5 \times$ fewer tokens in the same wall time due to per-head assembly overhead.

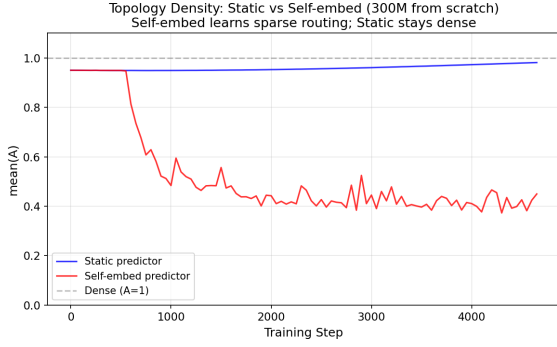


Figure 5: Topology density over training. Self-embed rapidly learns sparse routing (mean $\mathbf{A} \rightarrow 0.42$), while Static stays dense. **First successful topology learning** across all experiments.

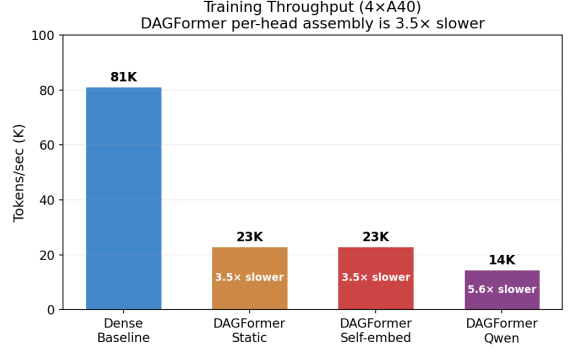


Figure 6: Training throughput ($4 \times A40$). Per-head input assembly makes DAGFormer $3.5 \times$ slower; Qwen adds another $1.6 \times$. Root cause of the NLL gap.

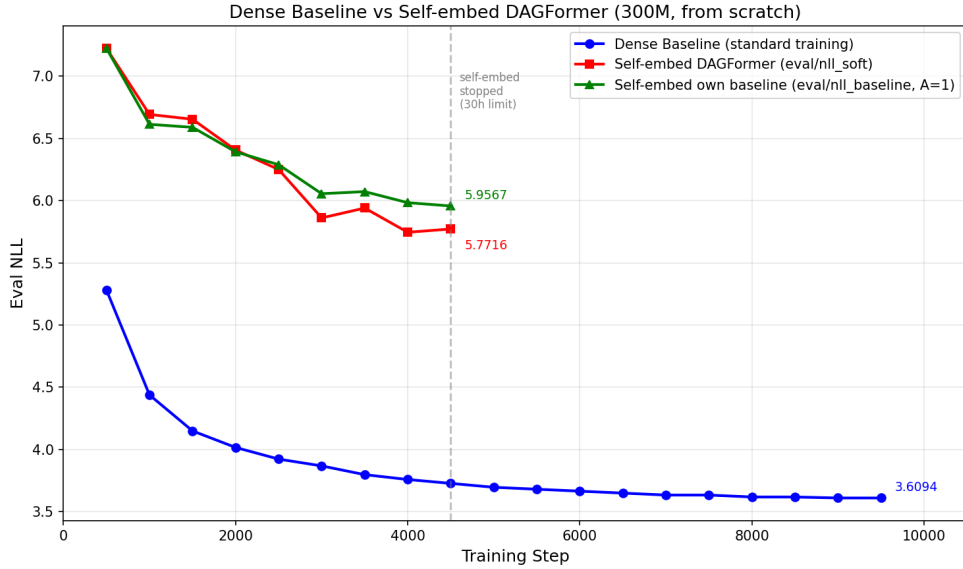


Figure 7: Eval NLL comparison. Blue: dense baseline. Red: self-embed DAGFormer (eval/nll_soft). Green: self-embed’s own dense forward ($\mathbf{A} = 1$). Key: red $<$ green after step 3000 — learned topology helps the same undertrained model, but both are far behind the dense baseline due to throughput difference.

Key findings.

- **Self-embed learns sparse topology** (mean $\mathbf{A} = 0.42$). First experiment where the predictor meaningfully deviates from dense.
- **Learned topology helps**: soft NLL (5.75) $<$ own-baseline NLL (5.96), a 0.21 nat improvement on the same undertrained model.
- **Static predictor fails**: mean $\mathbf{A} = 0.98$, predictor gradient ~ 0 . Without input-dependent signal, the predictor cannot learn.
- **$3.5 \times$ throughput overhead is the bottleneck**: the model sees far fewer tokens, so the base model is severely undertrained.

4 Current Experiments (In Progress)

To address the throughput bottleneck, we are testing five new strategies.

4.1 Addressing Throughput

Staged training. Load a fully-trained baseline checkpoint (10K steps, NLL=3.609), then switch to DAGFormer forward for 2,000 additional steps. The base model starts with good language modeling; only needs to adapt to routing. Predictor starts fresh.

Alternating training. Train from scratch with a 9:1 ratio of standard (fast) to DAGFormer (slow) steps. Both update the base model; the predictor only updates on DAGFormer steps. Expected throughput $\approx 90\%$ of baseline.

Self-embed without detach. Same as staged, but gradients from the predictor flow back into the embedding layer (`.detach()` removed). The embedding learns to produce representations useful for both language modeling and topology prediction.

4.2 Improving Predictor Input

The original self-embed applies mean pooling over raw token embeddings — essentially bag-of-words, losing all positional and syntactic information.

Mini-encoder (independent). Separate embedding table ($100K \times 256$) + 2-layer transformer encoder (dim=256, 4 heads). Learns its own contextual representation optimized for topology prediction. $\sim 41M$ parameters.

Context-embed (shared). Reuses the LLM’s embedding table, projects down ($1024 \rightarrow 256$), then applies a 2-layer transformer encoder. $\sim 16M$ parameters (no extra vocab cost). LLM embedding is detached.

#	Name	Strategy	Predictor Input	Steps
1	Staged	Baseline ckpt $\rightarrow$ DAGFormer	Self-embed (mean pool)	2,000
2	Alternating	9:1 standard:DAGFormer	Self-embed (mean pool)	10,000
3	No-detach	Baseline ckpt + no detach	Self-embed (unfrozen)	2,000
4	Mini-encoder	Baseline ckpt	Independent 2L transformer	2,000
5	Context-embed	Baseline ckpt	Shared embed + 2L transformer	2,000

Table 3: Five new experiments currently running. Key axes: staged vs. alternating, detach vs. no-detach, bag-of-words vs. contextual input, independent vs. shared embedding.

5 Discussion & Next Steps

5.1 The Core Tension

DAGFormer faces a fundamental speed–quality tradeoff. Per-head input assembly ($\text{input}_j = \text{base} + \sum_i \mathbf{A}[i, j] \cdot \text{head_out}_i$) requires layer-sequential einsum operations, making it $\sim 3.5\times$ slower than standard batched attention. More DAGFormer steps means better topology but

fewer total tokens and a worse base model. Our current experiments explore different points on this tradeoff.

5.2 Open Questions

1. **Does context-dependent topology matter?** Static failed, but may just need different optimization. If a fixed topology suffices, the predictor is unnecessary at inference.
2. **Can we amortize the overhead?** Sparse $\mathbf{A}$ (skip zero gates), fused kernels, or applying DAGFormer to only a subset of layers.
3. **What is the right predictor architecture?** Mean-pooled embeddings are too crude; a full Qwen encoder is too expensive. Mini-encoder and context-embed are middle ground — is 2 transformer layers enough?
4. **Optimal model scale?** 300M may be too small (fewer redundant heads). Would 1B+ show clearer benefits?
5. **Distillation?** Train a dense teacher, distill into a DAGFormer student with sparse topology for inference efficiency.

5.3 Proposed Future Recipes

Recipe	Description	Rationale
Aggressive alternating	50:1 or 100:1 standard:DAGFormer ratio	Near-baseline throughput; predictor gets minimal but possibly sufficient signal
Layer-subset DAGFormer	Apply $\mathbf{A}$ only to last 4 layers	Reduce overhead to $\sim 1.5\times$; early layers may not need routing
Sparse $\mathbf{A}$ forward	Skip einsum for $\mathbf{A}[i, j] < 0.01$	Exploit learned sparsity for speed (mean $\mathbf{A} = 0.42 \Rightarrow 58\%$ skippable)
Distillation	Dense teacher $\rightarrow$ sparse student	Decouple topology learning from language modeling
Curriculum	Short seq (256) first, then increase	Faster DAGFormer at small seq_len
Scale up	1B model from scratch	More heads = more redundancy = more room for topology

Table 4: Proposed future recipes for team discussion.

5.4 Summary

We have established that: (1) midtraining topology onto pretrained models does not work — the loss landscape is flat at $\mathbf{A} = \mathbf{1}$; (2) pretraining from scratch enables topology learning (self-embed achieves mean $\mathbf{A} = 0.42$ with sparse routing that outperforms its own dense baseline); (3) the main bottleneck is the $3.5\times$ throughput overhead. Five strategies are currently under evaluation, and additional recipes are proposed above for team discussion.